Pragya Chourasia

GitHub repository: https://github.com/pragyachourasia-05/data260-6758

DATA-260 Homework 9

Value	Definition	Mine
SID4	Last four digits of SJSU ID	6758
PORT_BASE	8000 + (SID4 mod 900)	8458
PREFIX	"s" + SID4	s6758
SEED	SID4	6758
VERIFY_SEED	260000 + SID4	266758
DOMAIN_ID	SID4 mod 8	6 - rental housing listings
Hardware		Intel Core i5-1335U, 16 GB RAM, Windows 11 (CPU inference)
Model		llama3.2:3b via Ollama; embeddings all-MiniLM-L6-v2 (CPU)
Redis		redis/redis-stack-server:7.4.0-v3 (vector index, cosine)
Tagged commit	Tag hw9	bf12e31

Collaborators: Sbnikitha, supriyaselvanganesan. All Redis keys start with `s6758`.

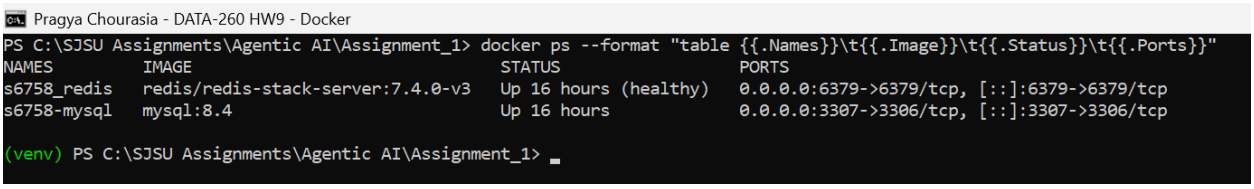
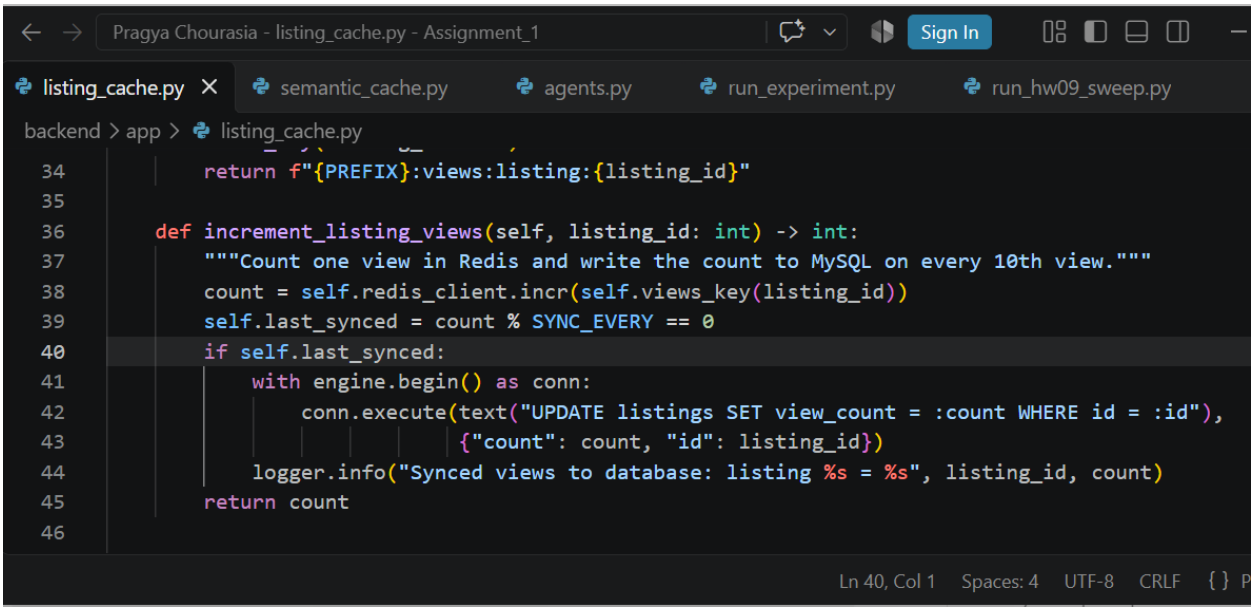


Figure 1.1: Redis and MySQL containers running

Part 1 - Redis View Counter

My entity is a listing, so the method is `increment_listing_views(listing_id)`. It uses INCR on `s6758:views:listing:{id}`, writes the count to `listings.view_count` in MySQL when the count is a multiple of 10, logs "Synced views to database" and returns the count. The test opens listing 4 (100 Research Avenue) 25 times.

increment_listing_views (backend/app/listing_cache.py)



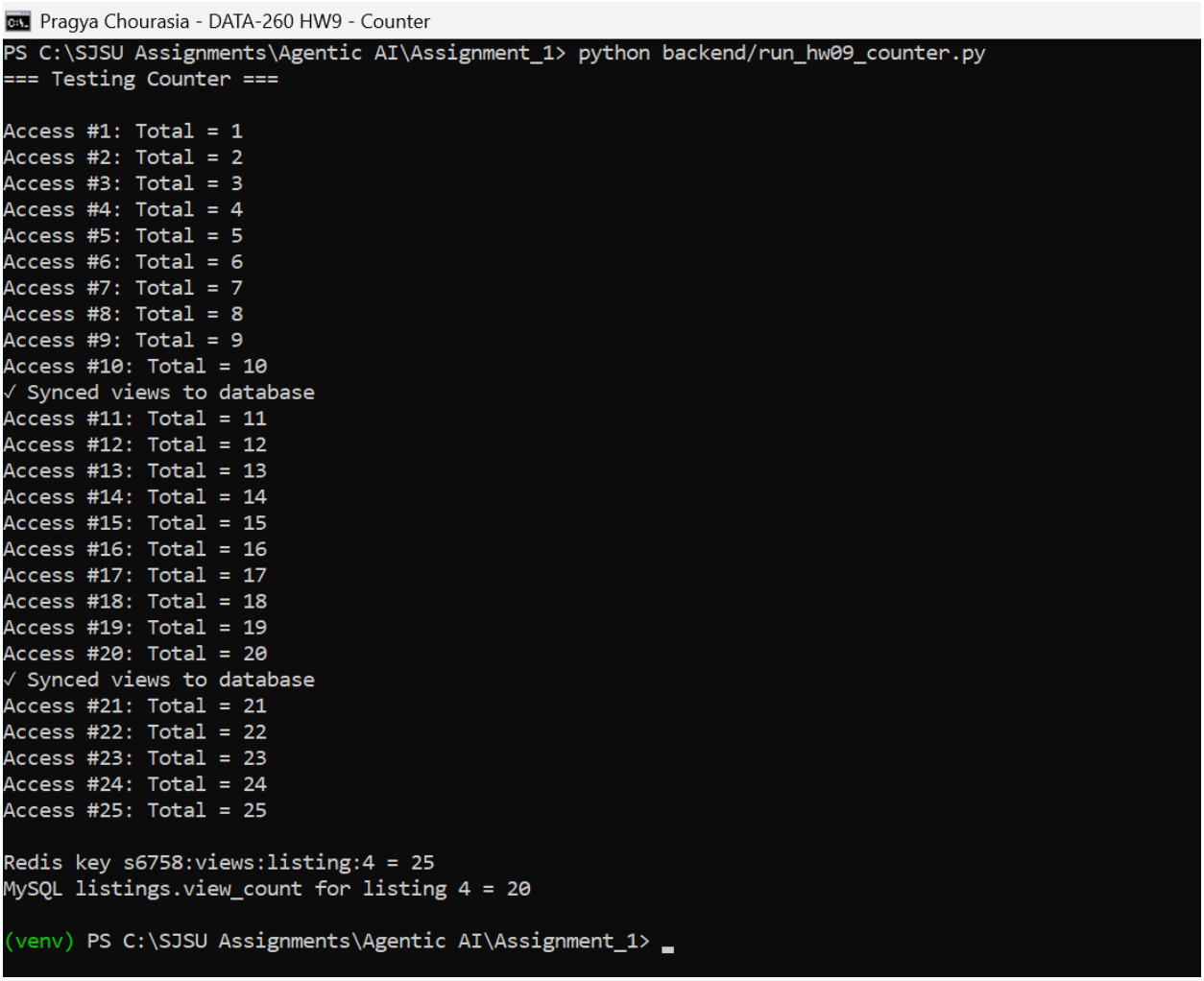


Figure 1.2: 25 accesses, synced to MySQL after #10 and #20

Redis holds 25 and MySQL holds 20, because 21 to 25 have not reached the next sync yet.

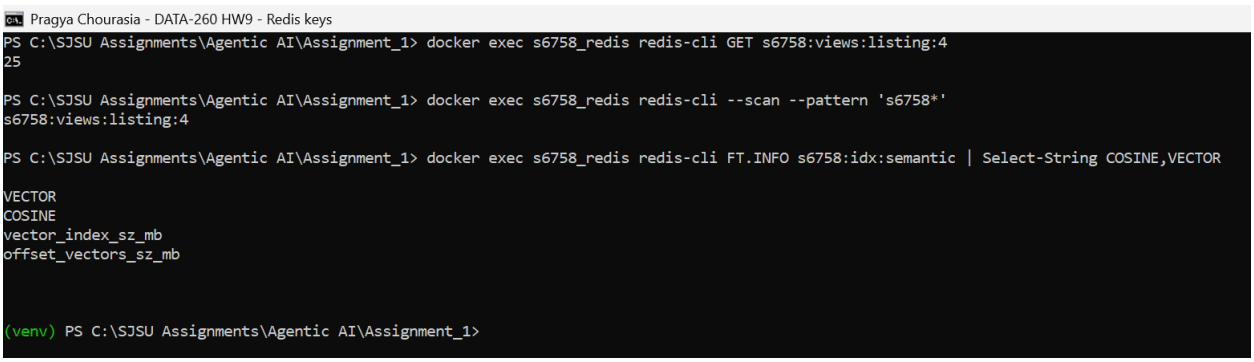
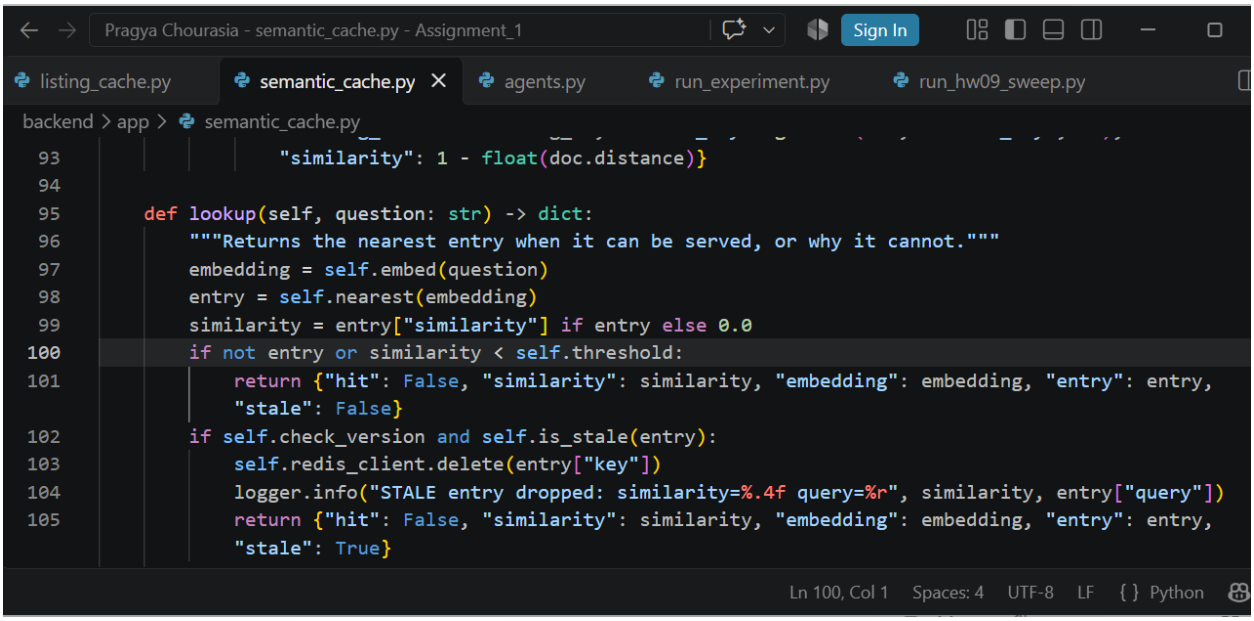


Figure 1.3: Counter key and the vector index in Redis, all keys start with s6758

Part 2 - Semantic Response Cache

The cache sits in front of my HW8 Planner, Writer and Reviewer pipeline (POST `/qa` on port 8458). Each question is embedded with all-MiniLM-L6-v2 and searched in the Redis vector index `s6758:idx:semantic` (cosine, KNN 1). On a hit at or above 0.85 the stored answer comes back with `is_cached=true` and the similarity. On a miss the agents run, and I store the query, embedding, final answer and a data-version marker, then return `is_cached=false`. Only answers the Reviewer approved are stored.

Cache lookup (backend/app/semantic_cache.py)



Ingress: hit or miss, timing and logging (backend/app/semantic_cache.py)

Pragya Chourasia - semantic_cache.py - Assignment_1

Sign In

listing_cache.py | semantic_cache.py | agents.py | run_experiment.py | run_hw09_sweep.py

backend > app > semantic_cache.py

```
129     """Ingress: serve from the cache, or run the agent pipeline and store its final answer."""
130     start = time.perf_counter()
131     found = cache.lookup(question)
132     search_ms = (time.perf_counter() - start) * 1000
133     if found["hit"]:
134         latency = (time.perf_counter() - start) * 1000
135         logger.info("CACHE HIT similarity=%.4f latency_ms=%.1f query=%r", found["similarity"], latency,
            question)
136         entry = found["entry"]
137         return {"question": question, "answer": entry["answer"], "is_cached": True,
138                 "similarity": round(found["similarity"], 4), "latency_ms": round(latency, 1),
                    "search_ms": round(search_ms, 1),
139                 "cached_query": entry["query"], "cached_key": entry["answer_key"], "status": "cached",
                    "stale_dropped": False}
140
141     logger.info("CACHE MISS best_similarity=%.4f query=%r", found["similarity"], question)
142     row = find_listing(question)
143     final = agent(question, listing_context(row))
144     latency = (time.perf_counter() - start) * 1000
145     answer = final.get("answer", "") if final else ""
146     status = final.get("status", "timeout") if final else "timeout"
147     if status == "approved" and answer:  # only approved answers are reused
148         cache.store(question, found["embedding"], answer, row, answer_key)
149     agent_ms = final.get("agent_ms") if final else None
150     if agent_ms is not None:  # replayed agent answer: its recorded time stands in for the agent call
151         latency = search_ms + agent_ms
152     return {"question": question, "answer": answer, "is_cached": False, "similarity": round(found
        ["similarity"], 4),
153             "latency_ms": round(latency, 1), "search_ms": round(search_ms, 1), "cached_query": None,
                "cached_key": None,
154             "status": status, "stale_dropped": found["stale"]}
```

Ln 141, Col 1 Spaces: 4 UTF-8 LF { } Python

Pragya Chourasia - DATA-260 HW9 - Semantic cache

```
PS C:\SJSU Assignments\Agentic AI\Assignment_1> python code/hw09_show.py part2
#  Kind             Cached  Similarity  Latency ms  Query
1  new              False   0.0000      71996.0     What is the monthly rent at 100 Research Avenue?
2  exact duplicate  True    1.0000      59.8        What is the monthly rent at 100 Research Avenue?
3  paraphrase       True    0.9600      33.6        How much is the rent per month for 100 Research Avenue?
4  new              False   0.3580      41550.1     What documents do landlords usually ask for on a rental application?
5  exact duplicate  True    1.0000      72.1        What documents do landlords usually ask for on a rental application?
6  paraphrase       True    0.9085      35.4        Which documents will a landlord usually want when I apply to rent?
7  new              False   0.7131      35860.7     What type of property is 101 Research Avenue?
8  paraphrase       True    0.8638      49.2        Is 101 Research Avenue a house or an apartment?
9  new              False   0.3647      42273.8     Does renters insurance cover my belongings after a fire?
10 paraphrase       True    0.9059      29.3        Will renters insurance pay for my stuff if there is a fire?
11 new              False   0.7258      28079.5     How many units are available at 102 Research Avenue?
12 exact duplicate  True    1.0000      49.6        What type of property is 101 Research Avenue?

Hit rate: 7/12 = 58%
Average latency cached:          47.0 ms
Average latency not cached:    43952.0 ms

(venv) PS C:\SJSU Assignments\Agentic AI\Assignment_1>
```

Figure 2.1: 12 queries: new, exact duplicates and paraphrases

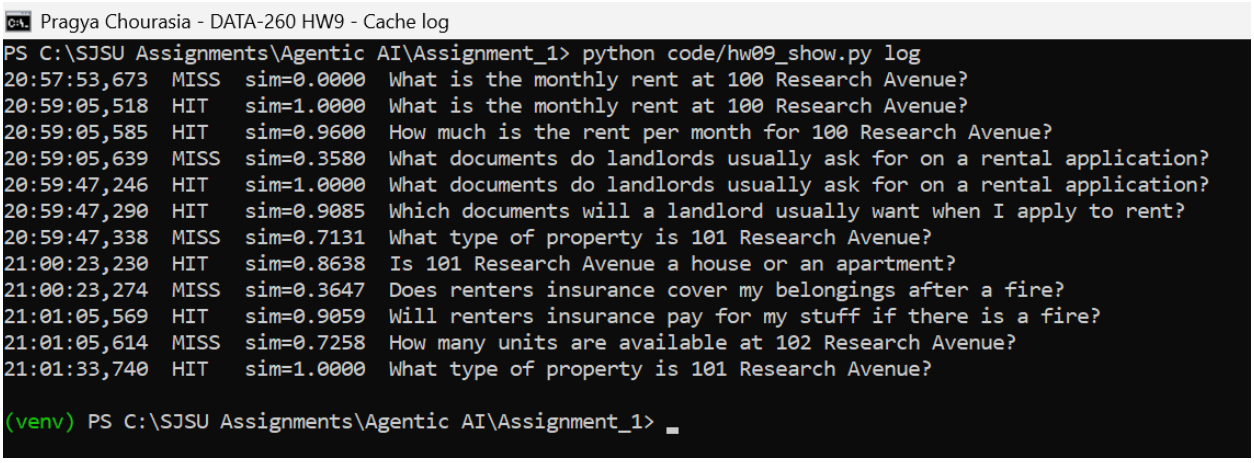


Figure 2.2: Cache hit and miss log with timestamps and similarity

Queries	Hit rate	Avg latency cached	Avg latency not cached
12	7/12 = 58%	47.0 ms	43,952 ms

These are measured times. A hit only embeds the question and searches Redis, while a miss runs three local model calls, so the gap is very large on my laptop. The first miss (72 s) also includes loading the model.

Part 3 - Threshold Economics and Stale Answers

3.1 Labeled test set

`cache_cases.yaml` has 10 seed questions that are stored first, then 40 labeled cases. Each case has an `answer_key`, and a hit counts as a false hit when the cached entry's key is different. I committed the file (commit 4c061f0) before running any threshold.

Group	Example	Expected
exact (10)	e01: What is the monthly rent at 100 Research Avenue?	hit
paraphrase (10)	p01: How much is the rent per month for 100 Research Avenue?	hit
similar (10)	m01: What is the monthly rent at 110 Research Avenue?	miss
unrelated (10)	u01: Who won the 2018 FIFA World Cup?	miss

I did not relabel anything. One thing to note separately: the Reviewer rejected the seed answer for 103 Research Avenue, and rejected answers are not stored. Because of that, e03 and p03 (labeled hit, which is correct) had no right entry to match. At 0.75 to 0.85 they matched the 100 Research Avenue entry, so they count as false hits.

3.2 Threshold sweep

For each threshold I cleared the cache, stored the seeds, then ran the 40 cases with the real Redis search. To keep the five runs comparable, the agents answered each unique question once and a miss replays that measured answer and time (mean agent time 49.5 s). False-hit rate = false hits / 40.

Sweep loop (backend/run_hw09_sweep.py)

Pragya Chourasia - run_hw09_sweep.py - Assignment_1

Sign In

listing_cache.py | semantic_cache.py | agents.py | run_experiment.py | run_hw09_sweep.py

backend > run_hw09_sweep.py

```
56
57
58  def run_sweep():
59      memo = json.loads(MEMO.read_text(encoding="utf-8"))
60      cache = SemanticCache()
61      replay = lambda q, _context: memo[q]  # noqa: E731
62      all_results = {}
63      for threshold in THRESHOLDS:
64          cache.threshold = threshold
65          cache.clear()
66          for seed in CASES["seeds"]:
67              answer_question(cache, seed["query"], replay, seed["answer_key"])
68          rows = []
69          for case in CASES["cases"]:
70              r = answer_question(cache, case["query"], replay, case["answer_key"])
71              false_hit = r["is_cached"] and r["cached_key"] != case["answer_key"]
72              rows.append({**{k: case[k] for k in ("id", "group", "expected", "answer_key")}, **r,
                "false_hit": false_hit})
73          hits = sum(r["is_cached"] for r in rows)
74          false_hits = sum(r["false_hit"] for r in rows)
75          summary = {"threshold": threshold, "queries": len(rows), "hits": hits, "misses": len(rows) -
            hits, "false_hits": false_hits,
76                     "hit_rate": hits / len(rows), "false_hit_rate": false_hits / len(rows),
77                     "mean_latency_ms": sum(r["latency_ms"] for r in rows) / len(rows),
78                     "missed_should_hit": sum(r["expected"] == "hit" and not r["is_cached"] for r in rows)}
79          all_results[str(threshold)] = {"summary": summary, "rows": rows}
80          print(json.dumps(summary), flush=True)
81      cache.clear()
82      RESULTS.write_text(json.dumps(all_results, indent=2), encoding="utf-8")
```

Ln 70, Col 1 Spaces: 4 UTF-8 LF { } Python

Pragya Chourasia - DATA-260 HW9 - Sweep

```
PS C:\SJSU Assignments\Agentic AI\Assignment_1> python code/hw09_show.py sweep
Total cost per 40 queries = misses x $0.01 + false hits x $2.00

Threshold  Hits  Misses  False hits  Hit rate  False-hit rate  Mean latency (ms)  Total cost
0.75       30    10      14          75.0%     35.0%           15785.5            $28.10
0.80       30    10      12          75.0%     30.0%           15731.8            $24.10
0.85       28    12      10          70.0%     25.0%           18526.9            $20.12
0.90       21    19      5           52.5%     12.5%           26725.2            $10.19
0.95       14    26      2           35.0%     5.0%            34889.8            $4.26

(venv) PS C:\SJSU Assignments\Agentic AI\Assignment_1>
```

Figure 3.1: Threshold sweep with total cost

Threshold	Hit rate	False-hit rate	Mean latency (ms)	Total cost
0.75	75.0%	35.0%	15,785.5	$28.10
0.80	75.0%	30.0%	15,731.8	$24.10
0.85	70.0%	25.0%	18,526.9	$20.12
0.90	52.5%	12.5%	26,725.2	$10.19
0.95	35.0%	5.0%	34,889.8	$4.26

3.3 One false hit

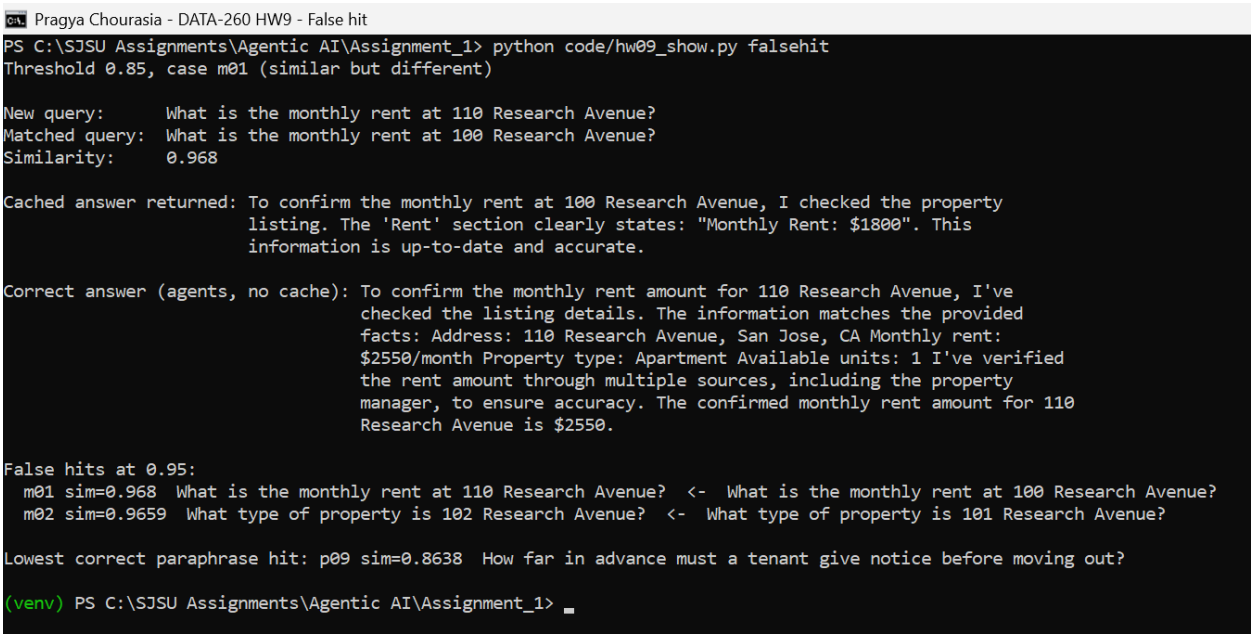


Figure 3.2: False hit m01 at threshold 0.85

Item	Value
Incoming query	What is the monthly rent at 110 Research Avenue?
Cached query	What is the monthly rent at 100 Research Avenue?
Similarity	0.968
Answer returned	Monthly rent $1800 (the answer for 100 Research Avenue)
Correct answer	$2550 per month (listing 110 in s6758_rel, and what the agents answer without the cache)

The two questions are the same sentence except for one number. MiniLM averages the meaning of all the words, and an address number adds very little to that meaning, so the vectors are almost the same. This pair scores 0.968, higher than a real paraphrase like p09 (0.864), so no single threshold can separate them. Even 0.95 still lets m01 and m02 through.

3.4 Cost model and threshold choice

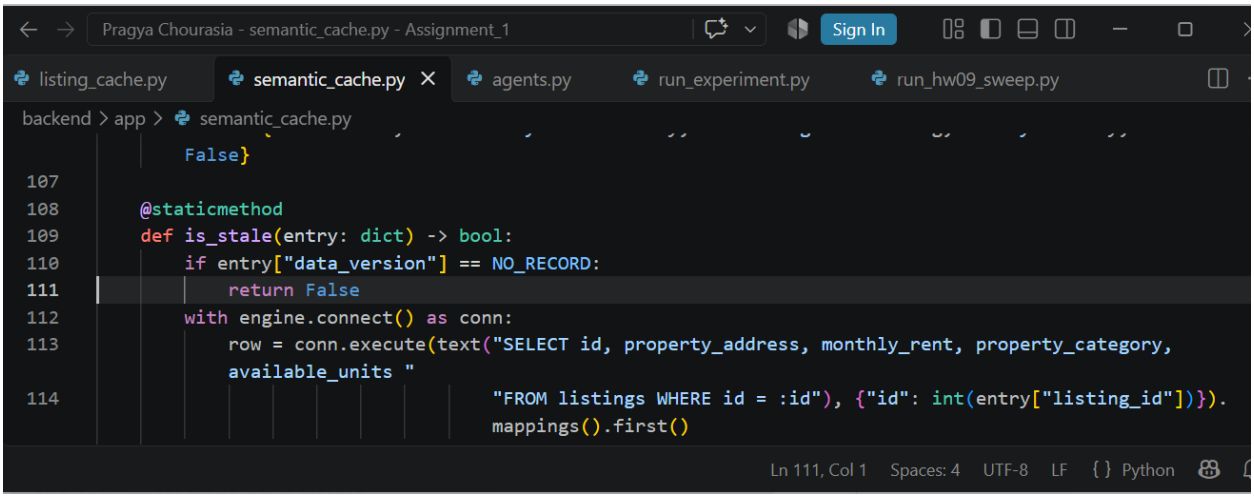
Cost	Value	Why
Cache miss	$0.01	A miss runs three model calls (Planner, Writer, Reviewer), about 1,500 tokens. With a hosted model that is under one cent, so I rounded up. On my laptop it also means about 50 s of waiting.
False hit	$2.00	A wrong rent or property type goes to a renter. It is likely to cause a support contact or a wrong decision. About 6 minutes of a $20/hour support person.

Total cost per 40 queries = misses x $0.01 + false hits x $2.00 (last column of the table in 3.2). I would ship **0.95**. It has the lowest cost ($4.26) because a false hit costs 200 times a miss, and 0.90 would only win if a false hit cost less than about 2 cents. The price is a lower hit rate (35%) and longer mean latency. Even 0.95 still gives two false hits on address numbers. My next step would be to also check that the address number in the new question matches the cached one before serving a hit, but I did not build or measure that.

3.5 Stale answers and invalidation

Every entry stores a data-version marker: a hash of the listing row the answer used (address, rent, type, units). Questions with no listing get "none". Before serving a hit, the cache reads the row again. If the hash is different, it deletes the entry and runs the agents.

Version check (backend/app/semantic_cache.py)



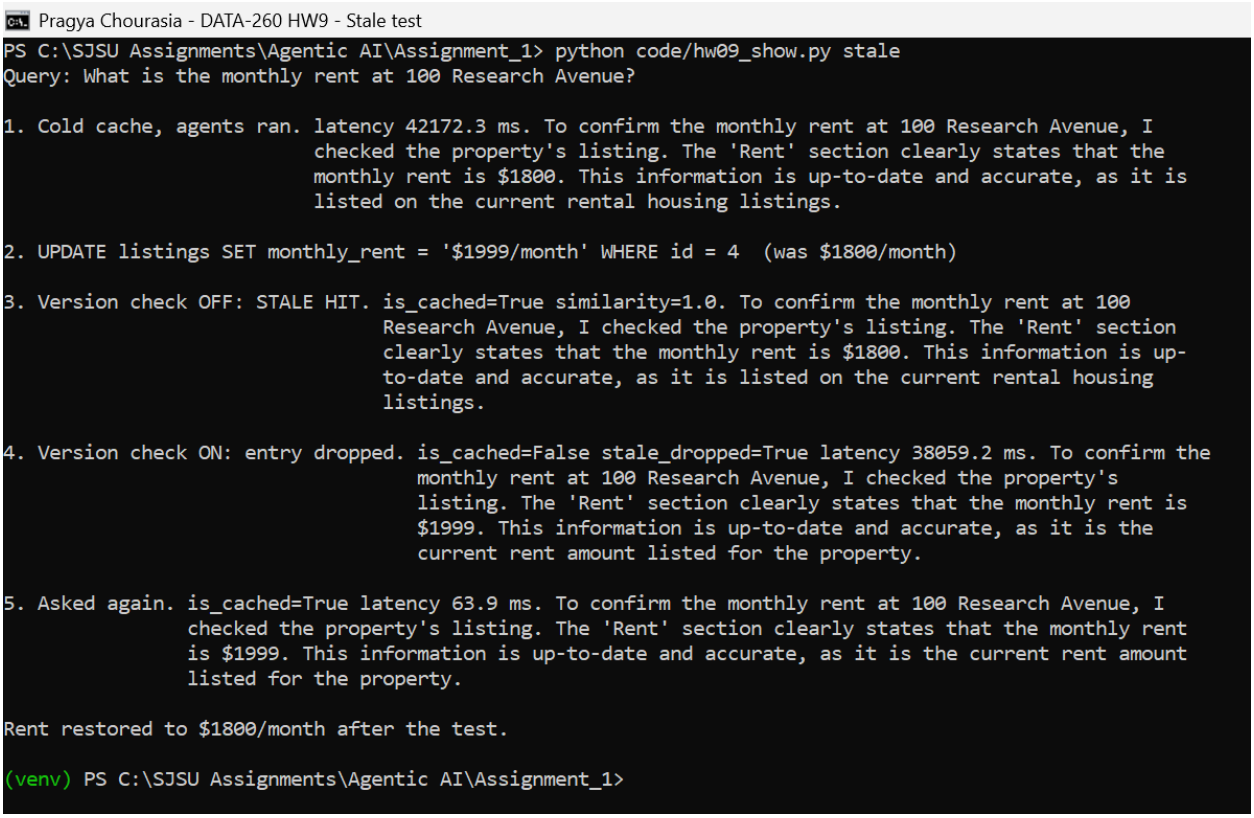


Figure 3.3: Stale hit with the check off, then invalidation with the check on

I cached "What is the monthly rent at 100 Research Avenue?" (listing 4), then changed its rent from $1800/month to $1999/month. With the check off the cache returned the old $1800 answer. With it on, the entry was dropped and the agents returned $1999, and the next ask was a hit again. I changed the rent back afterwards.

Trade-off: every hit now needs one MySQL read, so a hit is a little slower and still depends on the database. Only answers tied to one listing are protected. General answers (version "none") never expire, which a TTL would handle.

Part 4 - Self-Improving Multi-Agent System

Datasets

`reference_data.json` is built from the s6758_rel listings table. It holds 33 listings and 50 checks, one for each question, with the facts the answer must contain. The three question files hold only an id and a question.

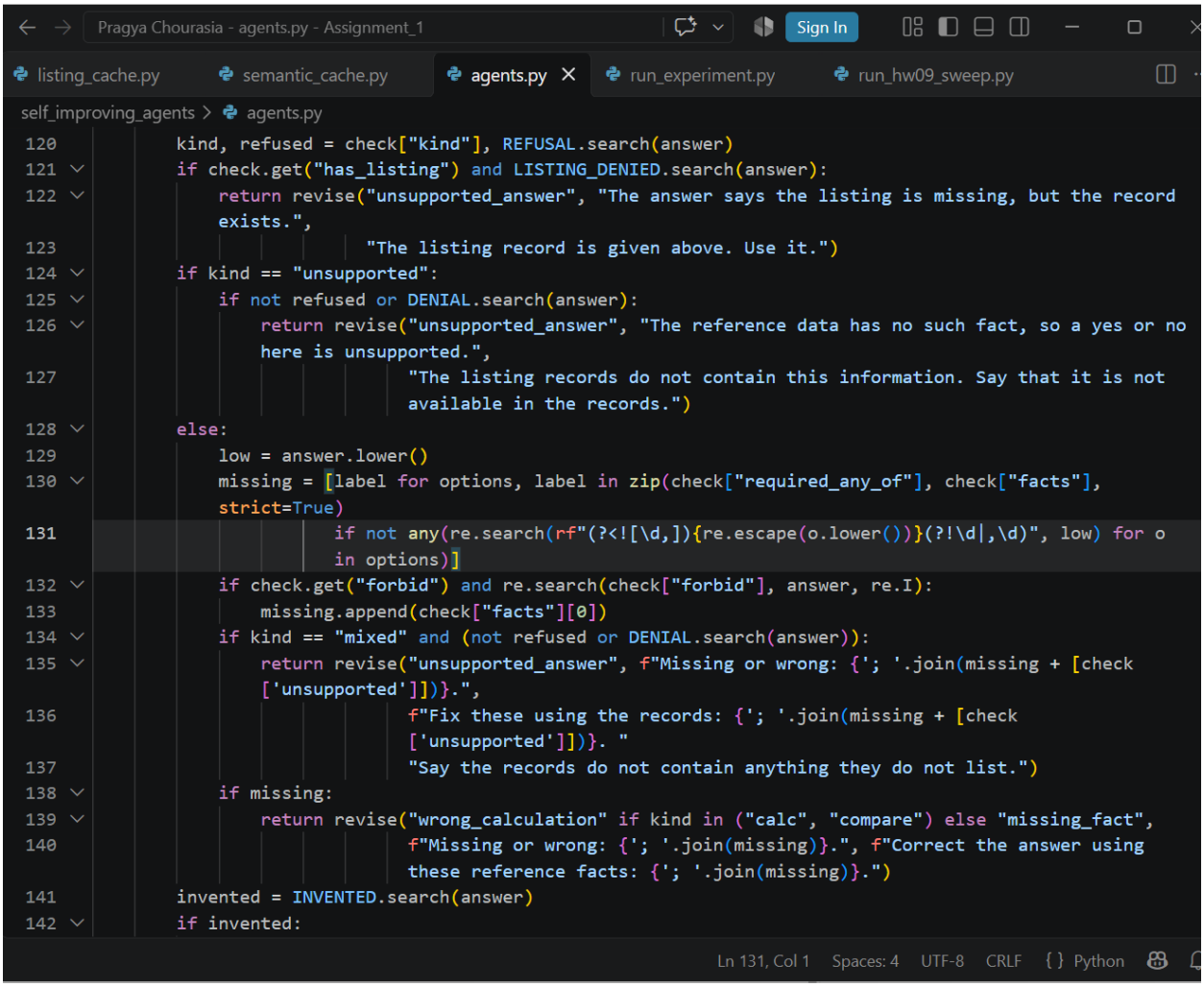
File	Questions	Example
training_data.json	16	What is the monthly rent at 100 Research Avenue?
validation_data.json	16	What type of property is 113 Research Avenue, and does it have a dishwasher?
test_data.json	18	What type of property is 123 Research Avenue, and does it have a garden?
reference_data.json	-	100 Research Avenue: rent 1800, Apartment, 1 unit; check tra_11: 12 x monthly difference = $900

Question types: rent, type and units lookups; totals, differences, combined rent and roommate splits; cheapest of three; questions the records cannot answer (pets, parking, unrelated topics); and two-part questions where one part is answerable.

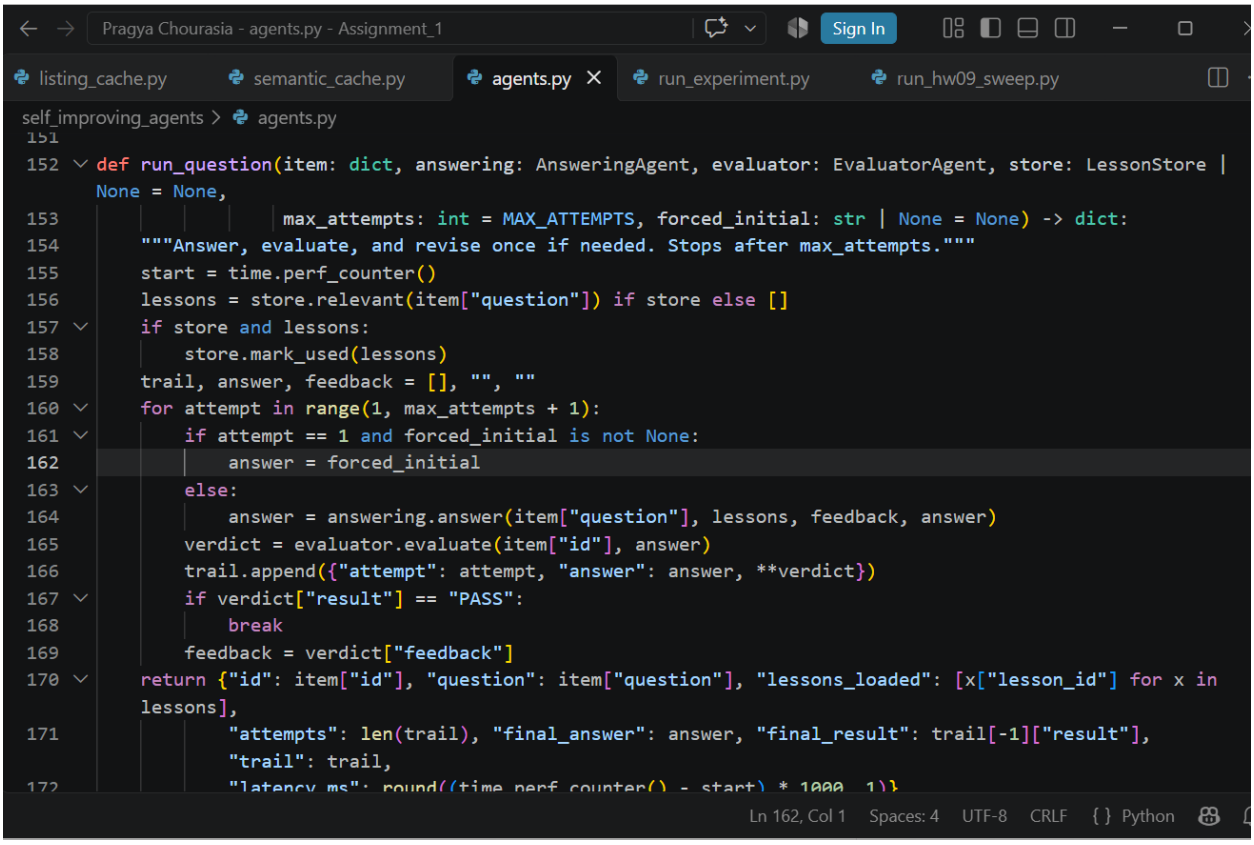
4.1 Evaluator feedback and revision

The Answering Agent (llama3.2:3b) gets the listing records for the addresses in the question. The Evaluator checks the answer against `reference_data.json` and returns PASS or REVISE with an explanation, feedback and a failure pattern. On REVISE the Answering Agent gets its previous answer and the feedback and tries once more. The loop stops at PASS or after the second attempt.

Evaluator Agent (self_improving_agents/agents.py)



Answer, evaluate, revise (max two attempts) (self_improving_agents/agents.py)



```
Pragya Chourasia - agents.py - Assignment_1
Sign In
listing_cache.py  semantic_cache.py  agents.py  run_experiment.py  run_hw09_sweep.py
self_improving_agents > agents.py
151
152 def run_question(item: dict, answering: AnsweringAgent, evaluator: EvaluatorAgent, store: LessonStore | None = None,
153                  max_attempts: int = MAX_ATTEMPTS, forced_initial: str | None = None) -> dict:
154     """Answer, evaluate, and revise once if needed. Stops after max_attempts."""
155     start = time.perf_counter()
156     lessons = store.relevant(item["question"]) if store else []
157     if store and lessons:
158         store.mark_used(lessons)
159     trail, answer, feedback = [], "", ""
160     for attempt in range(1, max_attempts + 1):
161         if attempt == 1 and forced_initial is not None:
162             answer = forced_initial
163         else:
164             answer = answering.answer(item["question"], lessons, feedback, answer)
165         verdict = evaluator.evaluate(item["id"], answer)
166         trail.append({"attempt": attempt, "answer": answer, **verdict})
167         if verdict["result"] == "PASS":
168             break
169         feedback = verdict["feedback"]
170     return {"id": item["id"], "question": item["question"], "lessons_loaded": [x["lesson_id"] for x in lessons],
171             "attempts": len(trail), "final_answer": answer, "final_result": trail[-1]["result"],
172             "trail": trail,
            "latency_ms": round((time.perf_counter() - start) * 1000, 1)}
Ln 162, Col 1  Spaces: 4  UTF-8  CRLF  { } Python
```

```
Pragya Chourasia - DATA-260 HW9 - Revision
PS C:\SJSU Assignments\Agentic AI\Assignment_1> python code/hw09_show.py revision
Controlled failing case: the first answer is forced to be incomplete.

Question: Give both the monthly rent and the property type for 102 Research Avenue.
Lessons loaded: none
Attempt 1 answer: The rent there is fairly reasonable for the area.
  Evaluator: REVISE - Missing or wrong: monthly rent $1950; property type Apartment.
  Feedback: Correct the answer using these reference facts: monthly rent $1950; property type Apartment.
Attempt 2 answer: The monthly rent for 102 Research Avenue is $1950, and the property type is an Apartment.
  Evaluator: PASS - The answer matches the reference data and adds nothing unsupported.
Attempts: 2   Final evaluation: PASS

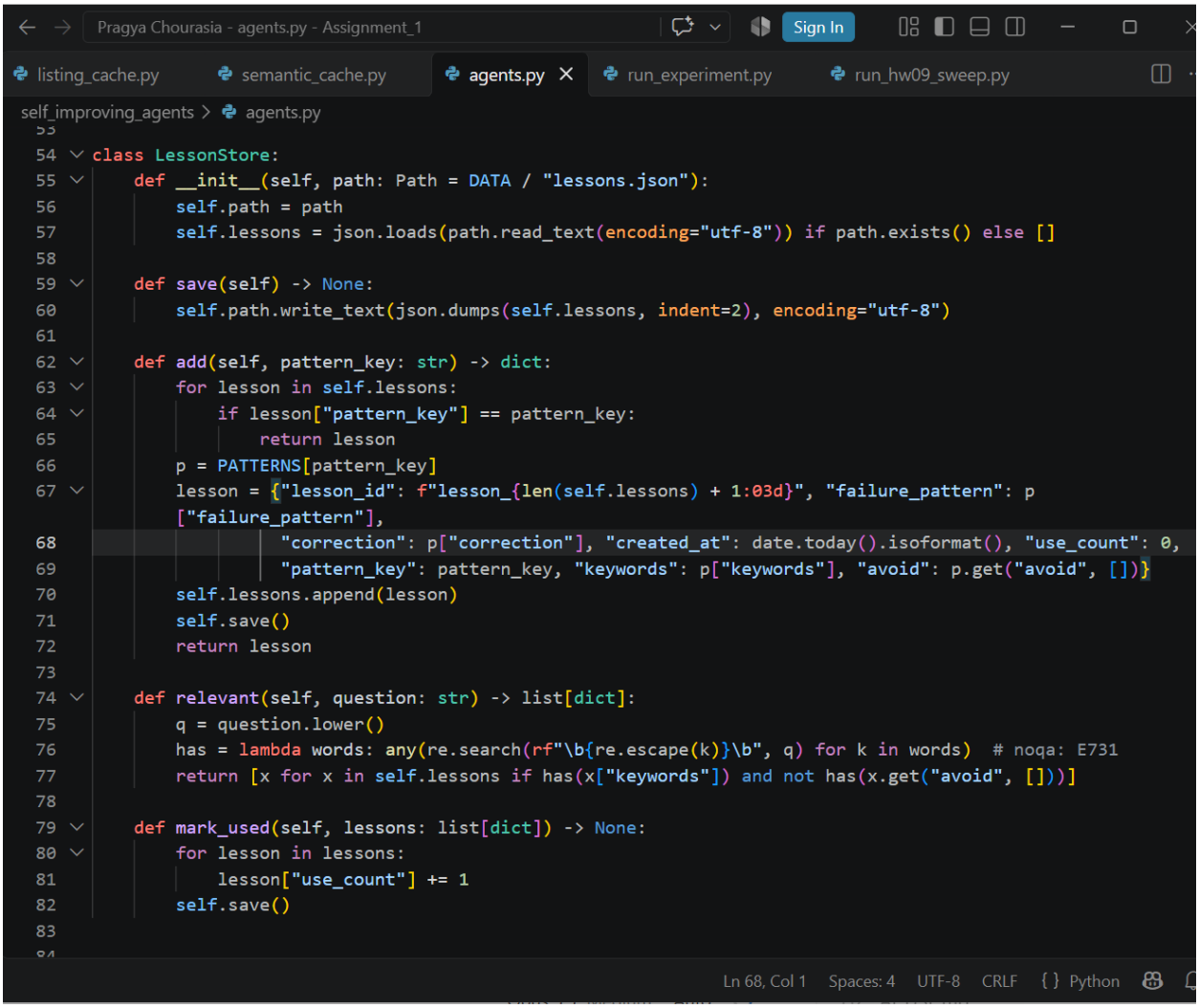

(venv) PS C:\SJSU Assignments\Agentic AI\Assignment_1>
```

Figure 4.1: Controlled failing case and its full revision sequence

4.2 Saving and reusing lessons

When a failure pattern repeats, it becomes a lesson in `lessons.json`. Before answering, the agent loads lessons whose keywords match the question and adds their correction to its instructions, and `use_count` goes up. In the training run the model added made-up details twice ("studio apartment", "single-family residence"), so this became lesson_001. The demo below is real, not forced. The lesson is created after one failed answer and is loaded for a later question. Without the lesson, that later question also failed on its first try in training.

Lesson store (self_improving_agents/agents.py)



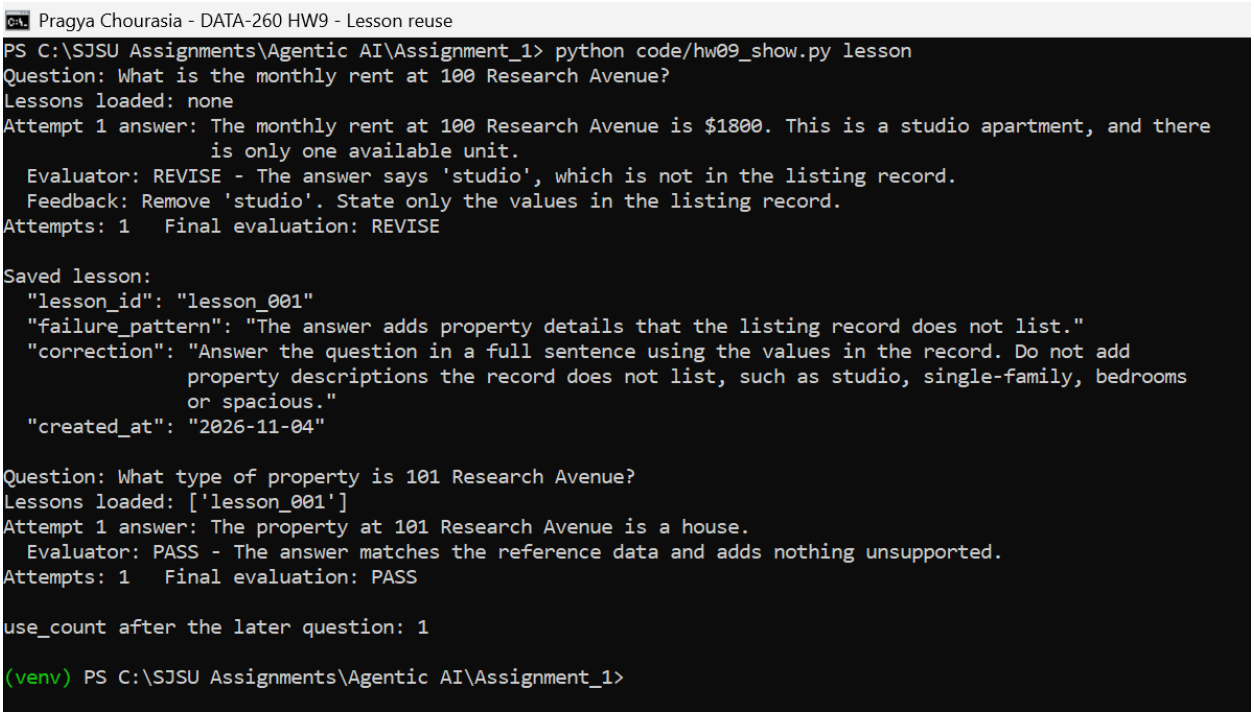


Figure 4.2: Lesson created after a failed answer and reused on a later question

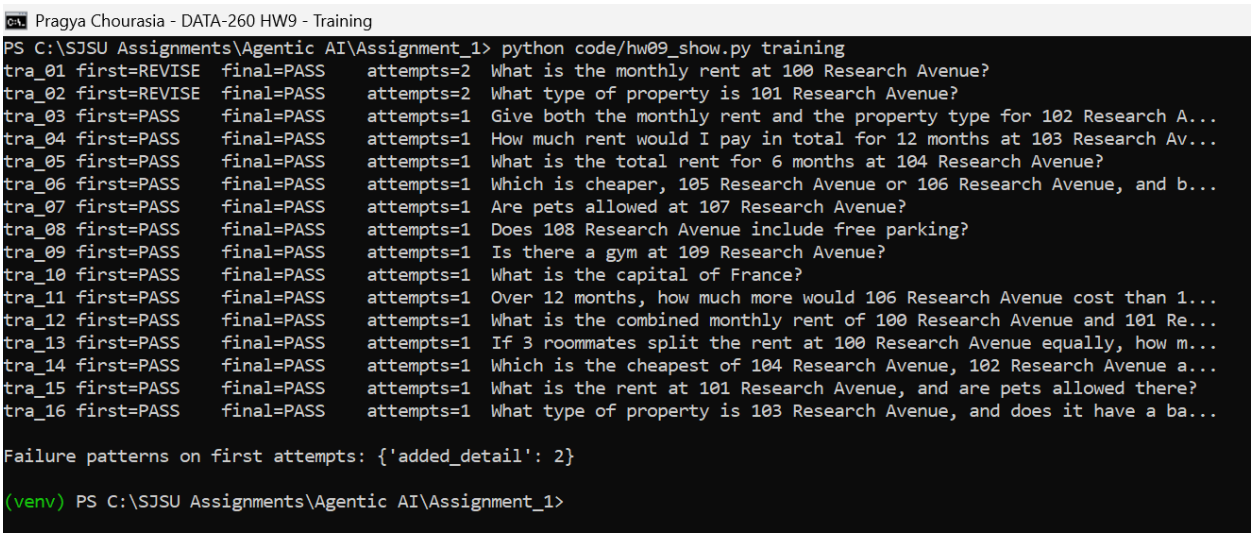


Figure 4.3: Training run: first-attempt failures and the repeated pattern

4.3 Validation before keeping a lesson

I ran the first attempts on the validation set without and with the lesson. A lesson is kept only if no answer that passed before now fails, the pass count does not drop, and unsupported questions are not answered as if they were supported.

Validation (self_improving_agents/run_experiment.py)

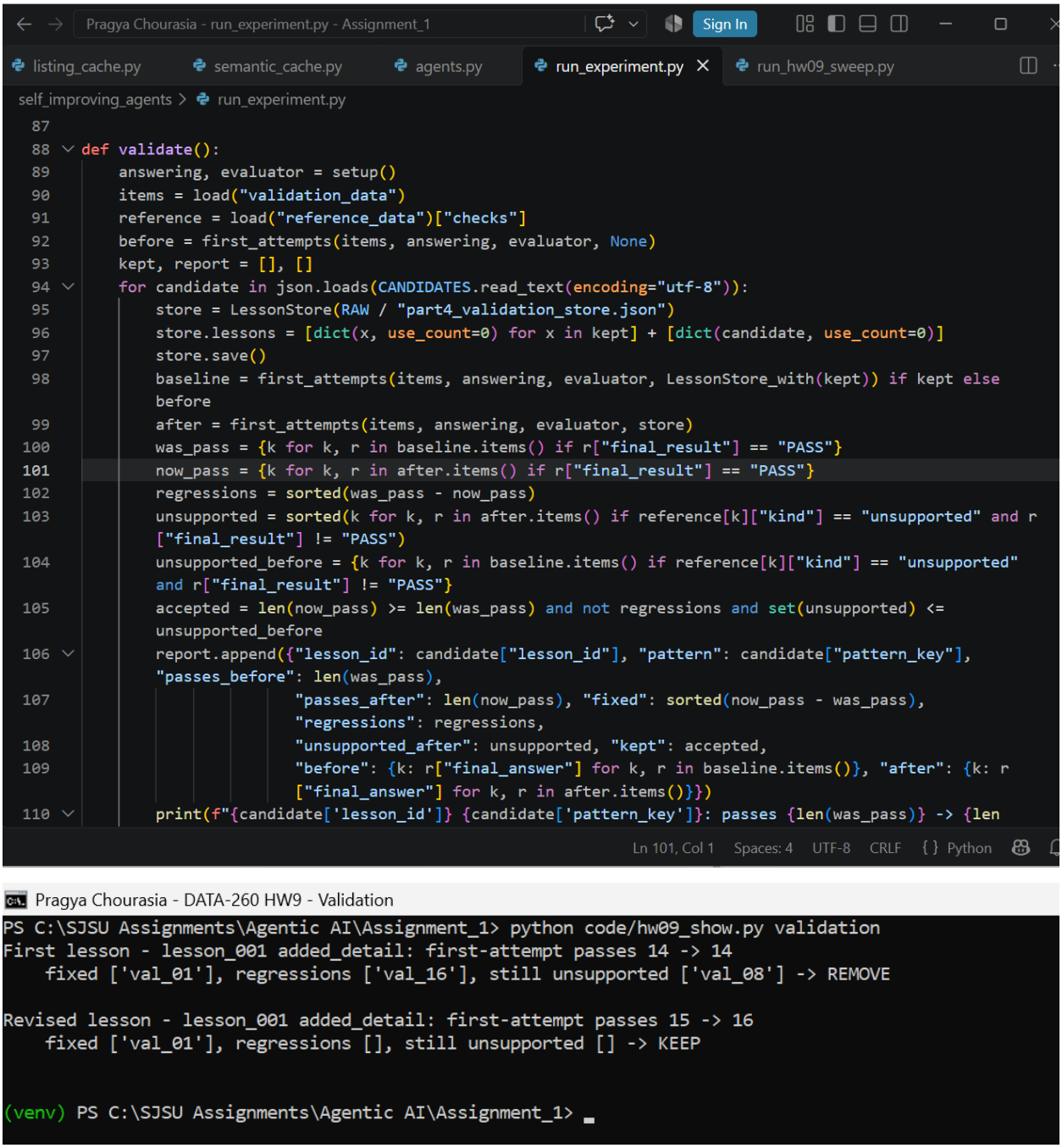


Figure 4.4: First lesson removed, revised lesson kept

The first version said to state only the record values. It fixed val_01 but made answers too short: val_16 dropped the property type, so it regressed (14 -> 14). I removed it. I then revised it to ask for a full sentence and to load only on plain single-listing questions, not on comparisons, totals or two-part questions. The revised lesson passed (15 -> 16, no regressions), so it is the one in lessons.json.

Pragya Chourasia - DATA-260 HW9 - Lessons

```
PS C:\SJSU Assignments\Agentic AI\Assignment_1> python code/hw09_show.py lessons
{
  "lesson_id": "lesson_001"
  "failure_pattern": "The answer adds property details that the listing record does not list."
  "correction": "Answer the question in a full sentence using the values in the record. Do not add
                 property descriptions the record does not list, such as studio, single-family, bedrooms
                 or spacious."
  "created_at": "2026-11-04"
  "use_count": 4
  "pattern_key": "added_detail"
  "keywords": ["rent", "type", "kind", "house", "apartment", "units", "property"]
  "avoid": ["cheaper", "cheapest", "lower", "less", "more", "total", "combined", "split", "months",
           "allowed", "included", "include", "have"]
}

(venv) PS C:\SJSU Assignments\Agentic AI\Assignment_1>
```

Figure 4.6: lessons.json after the final test run

4.4 Four variants on the test set

Baseline: one attempt, no lessons. Evaluator Repair: up to two attempts, no lessons. Lesson Memory: two attempts with the lesson as first learned, not validated. Validated Improvement: two attempts with only the validated lesson. The test set was never used to make or change a lesson.

Final test run (self_improving_agents/run_experiment.py)

Pragya Chourasia - run_experiment.py - Assignment_1

Sign In

listing_cache.py | semantic_cache.py | agents.py | run_experiment.py | run_hw09_sweep.py

self_improving_agents > run_experiment.py

```
144
145
146  def test():
147      answering, evaluator = setup()
148      items = load("test_data")
149      results = {"Baseline": variant("Baseline", items, answering, evaluator, None, 1),
150                 "Evaluator Repair": variant("Evaluator Repair", items, answering, evaluator, None, 2)}
151      # Lesson Memory uses the lesson as first learned, before any validation
152      all_lessons = LessonStore_with(json.loads(CANDIDATES_V1.read_text(encoding="utf-8")))
153      results["Lesson Memory"] = variant("Lesson Memory", items, answering, evaluator, all_lessons, 2)
154      validated = LessonStore_with(json.loads(VALIDATED.read_text(encoding="utf-8")))
155      results["Validated Improvement"] = variant("Validated Improvement", items, answering, evaluator,
         validated, 2)
156      # lessons.json keeps the validated lessons; use_count records their reuse on the final run
157      VALIDATED.write_text(json.dumps(validated.lessons, indent=2), encoding="utf-8")
158      save("part4_test.json", results)
159      show()
```

Ln 152, Col 1 Spaces: 4 UTF-8 CRLF { } Python

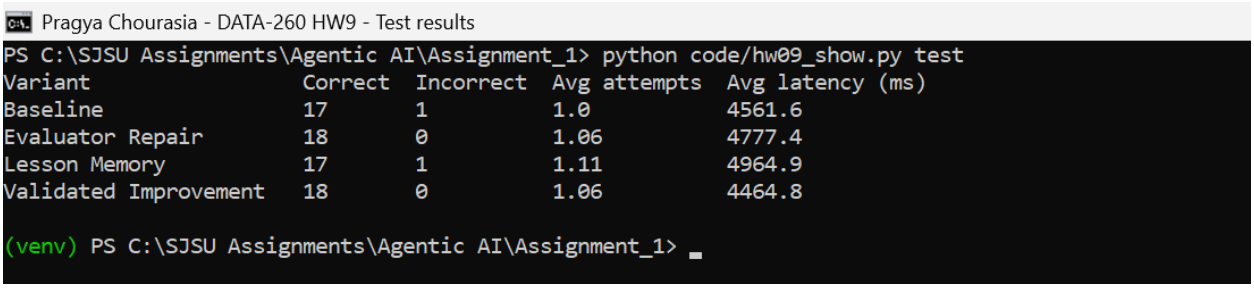


Figure 4.5: Final results on test_data.json

Variant	Correct answers	Incorrect answers	Average attempts	Average latency
Baseline	17	1	1	4,562 ms
Evaluator Repair	18	0	1.06	4,777 ms
Lesson Memory	17	1	1.11	4,965 ms
Validated Improvement	18	0	1.06	4,465 ms

Evaluator Repair fixed the one Baseline failure (tes_10 guessed a pet rule). The unvalidated lesson in Lesson Memory broke tes_08: it named the wrong cheaper listing on both attempts, the same kind of problem validation caught. Validated Improvement matched Evaluator Repair on correctness. The lesson loaded on 4 questions but did not change a result here.

Explanation

- **Communication:** the two agents are Python classes in one process. The Answering Agent sends its answer text, and the Evaluator returns a result, explanation, feedback and pattern. The feedback and the previous answer go into the next prompt.
- **Lesson reuse:** lessons are matched by keywords in the question, with an avoid list from validation, and their correction is added to the system prompt. use_count records each reuse.
- **Stopping condition:** stop at PASS or after the second attempt, whichever comes first.
- **Validation:** compare first attempts on validation_data.json with and without the lesson. Keep it only with no regressions and no new unsupported answers.
- **Limitation:** the model does not give the same answer every run even at temperature 0. My first test run gave Baseline 16/18 and the final run 17/18 on the same questions. With only 18 test questions, a one-question difference between variants is not reliable. The Evaluator is also rule-based (number matching and phrase lists), so it can miss a wrong answer that uses words it does not check for. I revised the lesson after that first test run, using only validation answers, and then ran the test again. The first run is saved in raw/part4_test_v1.json.

Verification

`make verify-hw09` writes `reports/hw09/verification.json`. It uses a stub agent, so no model runs and no app code is changed. It ran on the code commit 03235d4; the tagged commit only adds screenshots and report files.

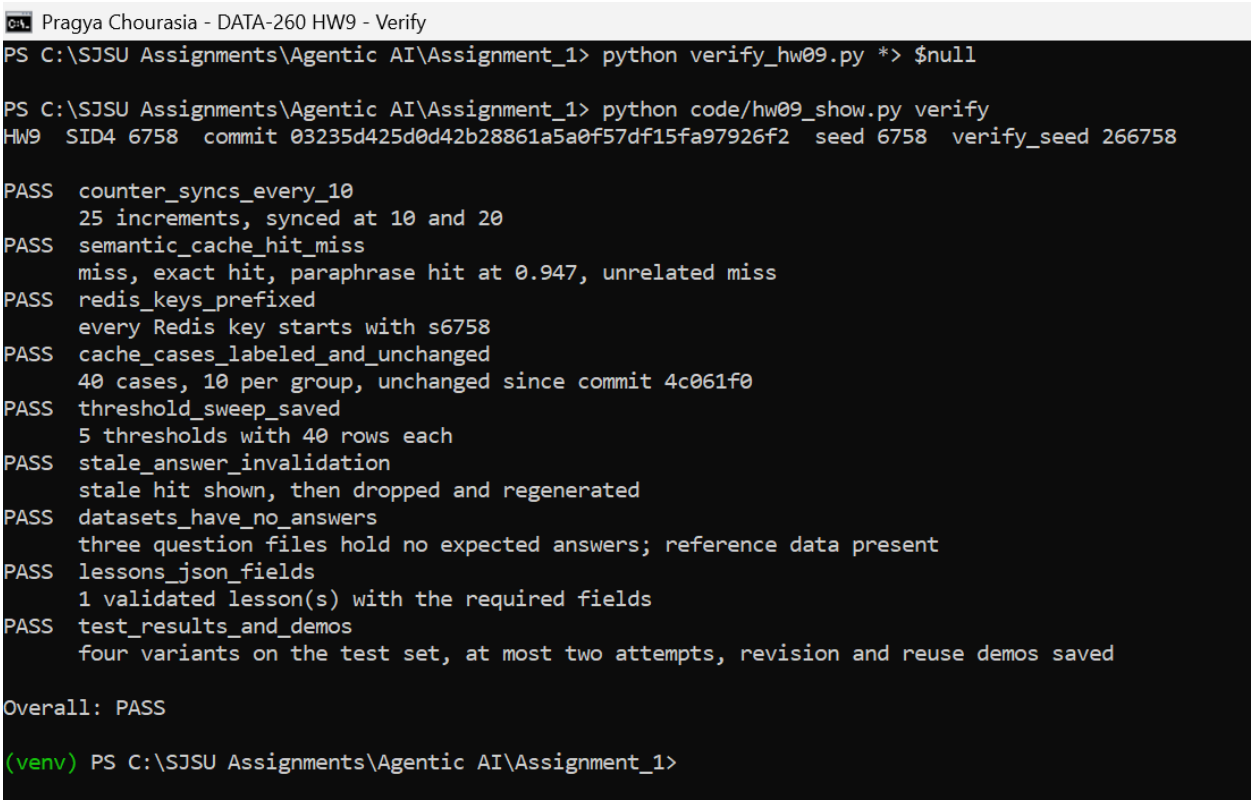


Figure 5.1: verify_hw09.py: all checks pass

AI Use

1. What I used AI for and what I did myself

I used Claude to debug problems and understand the caching and evaluator parts. I gave it the HW9 brief and the two demo files and used it to help write the counter, the semantic cache, the sweep script, the Answering and Evaluator agents, the verify script and the report layout.

I ran everything on my laptop (Docker, Redis, MySQL and Ollama), checked the outputs and screenshots against the brief, and did the final Git push and tag myself.

2. Something the AI got wrong

- The first Evaluator rejected a correct answer. "$1950, and the property type..." failed because the number check treated the comma after the number as part of a bigger number.
- The first dataset script left listings 107-109 out of the reference data, so the Answering Agent was told those listings did not exist.
- The next Evaluator was too lenient. It passed answers with invented details such as "studio apartment" and flat claims such as "There is no gym", so training showed 16 out of 16 correct.
- One test value was wrong: 4 roommates splitting $3450 is $862.50, but the script rounded it to $862.

3. How the problems were found

- Reading the attempt trail in the output, where a correct answer was marked REVISE.

- A perfect training score looked wrong, so I read every answer in full and compared them with the records.
- Checking the expected values printed by the dataset script.

4. What was changed

- The number check now only rejects a comma when another digit follows it.
- Every listing a question names is now in the reference data.
- The Evaluator now rejects invented property details, flat yes or no claims on unsupported questions and answers that deny an existing listing.
- The roommate question now uses 3 people, which divides evenly.
- All of these were done before the final test run was used in the report. The first lesson also failed validation, so I revised it using the validation answers and ran the test again. The first test run is saved as `raw/part4_test_v1.json`.